



Objetivos

- Comprender la definición y manejo de tipos y estructuras de datos en Haskell.
- Introducir los mecanismos de las clases y la programación genérica.

Tarea

En esta práctica, deberás programar en Haskell una serie de funciones que permitan trabajar con árboles binarios de búsqueda (https://en.wikipedia.org/wiki/Binary_tree).

Diseña e implementa en Haskell las funciones que se piden a lo largo de la práctica.

Todo el código deberá estar implementado en un módulo Haskell independiente y autocontenido, denominado '**BinaryTree**', para que pueda ser usado y probado desde un programa principal distinto al tuyo en la evaluación de la práctica.

1. Árboles binarios

Define en Haskell los **tipos** necesarios para implementar una estructura de datos que represente un árbol binario genérico pero homogéneo (es decir, todos los datos almacenados en el árbol son del mismo tipo, pero éste puede ser entero, real, cadena de caracteres, o incluso cualquier tipo de datos definido por el usuario).

Para poder construir un árbol de forma independiente del tipo de los datos contenidos en él, y de los detalles de implementación del propio árbol, se deberán implementar las siguientes funciones:

- **empty** – Esta función devuelve un árbol vacío, sin ningún elemento.
- **leaf x** – Esta función devuelve un árbol que consta de una única hoja, que contiene el elemento *x*.
- **tree x lc rc** – Esta función devuelve un árbol que contiene en la raíz el elemento *x*, con hijo izquierdo *lc* e hijo derecho *rc*. Tanto el hijo izquierdo como el hijo derecho son sub-árboles, no elementos individuales del tipo de datos genérico.
- **size t** – Esta función devuelve el número de elementos del árbol *t*.

También deberás implementar la **visualización** de un árbol por pantalla (el terminal), con la alineación y formato que estimes oportuno, pero de forma que quede claro el contenido del árbol –los datos– y su ubicación, y la forma del árbol. Esto debe ser posible simplemente utilizando la función de Haskell `print`. Para ello deberás hacer que tu árbol binario instancie la clase `Show`, para lo que necesitarás implementar la función `show`.

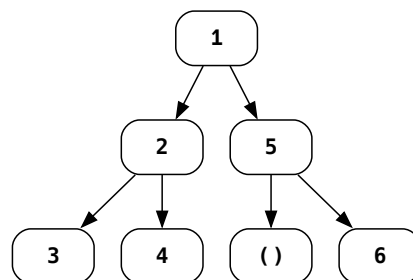
Para probar tu módulo *BinaryTree*, puedes o bien utilizar el intérprete de Haskell `ghci` o bien utilizar un programa principal. Prueba que el árbol es realmente genérico, es decir, comprueba que el árbol funciona con diferentes tipos de datos.

(continúa en la siguiente página...)

Mediante las funciones que has definido hasta el momento se puede construir y visualizar un árbol de forma manual, con código como el siguiente:

```
testTree = tree 1 (tree 2 (leaf 3) (leaf 4))
              (tree 5 empty (leaf 6))
main = print testTree
```

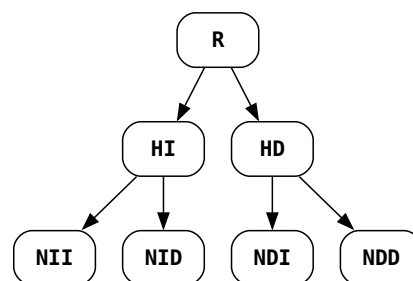
```
1
 \- 2
    \- 3
    \- 4
 \- 5
    \- ()
    \- 6
```



O bien:

```
testTree = tree "R"
              (tree "HI" (leaf "NII") (leaf "NID"))
              (tree "HD" (leaf "NDI") (leaf "NDD"))
main = print testTree
```

```
"R"
 \- "HI"
    \- "NII"
    \- "NID"
 \- "HD"
    \- "NDI"
    \- "NDD"
```



La visualización textual del árbol presentada se muestra como ejemplo.

2. Árboles binarios de búsqueda (ABB)

Con las funciones que has definido hasta el momento en tu módulo, el proceso para construir un árbol es tedioso y no garantiza que el árbol esté ordenado como requiere un árbol binario de búsqueda (ABB).

Implementa una función para añadir elementos a un árbol, de tal forma que se inserten en la posición adecuada, según el criterio de ordenación para el ABB:

- **add t x** – Añade el elemento x al árbol t, devolviendo el árbol binario de búsqueda resultante. Para nuestro caso, el criterio de ordenación se implementa simplemente mediante el uso de los operadores '<' y '>' (o bien '<=' y '>='), por lo que deberán ser aplicables al tipo de datos contenido en el árbol. Considera que el árbol binario de búsqueda puede contener valores repetidos, pero el valor en cada nodo siempre será estrictamente mayor que todos los valores de los elementos del subárbol de la izquierda, y menor o igual que todos los elementos del subárbol de la derecha.

Utilizando esta función, implementa otra función que construya un árbol binario de búsqueda a partir de los elementos de una lista:

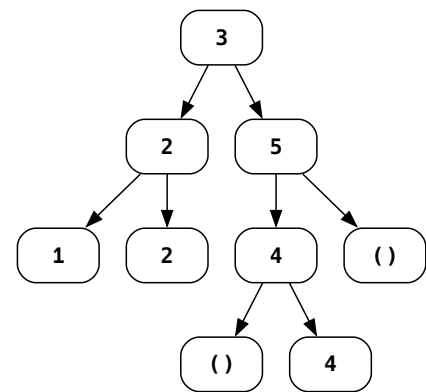
- **build xs** – Construye un árbol binario de búsqueda, comenzando con un árbol vacío e insertando sucesivamente los elementos de la lista xs.

(continúa en la siguiente página...)

Ahora deberías poder construir un árbol de la siguiente forma:

```
build [3, 2, 2, 5, 1, 4, 4]
```

```
3
 \- 2
    \- 1
       \- 2
    \- 5
       \- 4
          \- ()
             \- 4
                \- ()
```



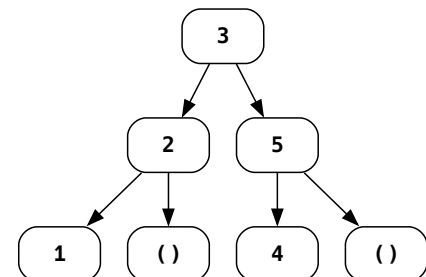
Para poder implementar estas funciones, dado que el árbol binario de búsqueda es genérico, necesitarás asegurar que el tipo de dato contenido en el árbol tenga definidos los operadores de comparación. Esto se consigue añadiendo una restricción para que dicho tipo deba ser instancia de la clase **Ord** de Haskell.

3. Árboles binarios de búsqueda equilibrados

El proceso anterior genera un árbol que, aun siendo un árbol binario de búsqueda correcto, puede presentar una distribución y altura que dependen del orden en que se den los elementos a insertar:

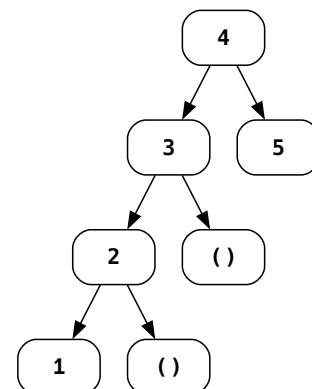
```
build [3, 2, 5, 1, 4]
```

```
3
 \- 2
    \- 1
       \- ()
 \- 5
    \- 4
       \- ()
```



```
build [4, 3, 2, 5, 1]
```

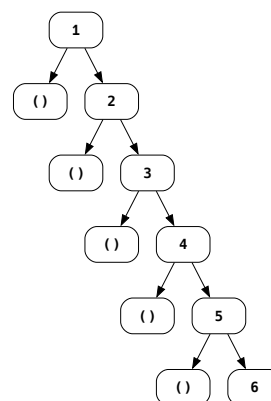
```
4
 \- 3
    \- 2
       \- 1
          \- ()
 \- ()
 \- 5
```



El caso peor (caso *patológico*) aparecería al utilizar una lista ordenada para construir el árbol, lo que daría como resultado un árbol degenerado:

```
build [1..6]
```

```
1
 \- ()
 \- 2
    \- ()
    \- 3
       \- ()
       \- 4
          \- ()
          \- 5
             \- ()
             \- 6
```



Implementa la siguiente función:

- **buildBalanced xs**: construye un árbol equilibrado a partir de los elementos de una lista, mediante el siguiente proceso:

- se ordena la lista 'xs'
- se selecciona el valor central (la mediana)
- se divide la lista en dos partes usando ese valor: la primera contiene los elementos menores que dicho valor, y la segunda los mayores o iguales.

Ten en cuenta que si hay valores repetidos, puede haber datos antes del elemento central, con su mismo valor, y que deben ir al sub-árbol derecho, por lo que las dos partes no tendrán la misma longitud.

- se contruyen los dos sub-árboles correspondientes a cada mitad, de forma recursiva

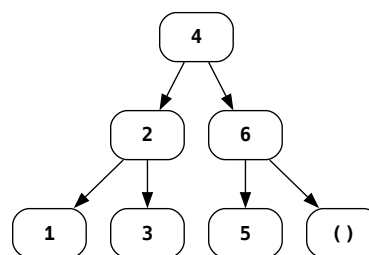
Esta función deberá construir un ABB equilibrado, independientemente del orden de los datos en la lista de entrada:

```
buildBalanced [1..6]
```

```
4
 \- 2
    \- 1
    \- 3
 \- 6
    \- 5
    \- ()
```

```
buildBalanced [2,4,5,1,6,3]
```

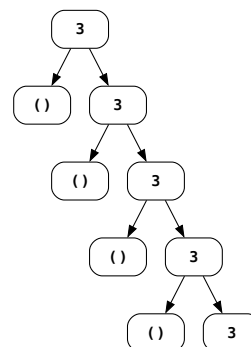
```
4
 \- 2
    \- 1
    \- 3
 \- 6
    \- 5
    \- ()
```



Nota: si existen valores repetidos en la lista, el ABB obtenido no tiene porqué resultar de altura mínima:

```
buildBalanced [3,3,3,3,3]
```

```
3
 \- ()
 \- 3
    \- ()
    \- 3
       \- ()
       \- 3
          \- ()
          \- 3
             \- ()
             \- 3
```



Para la implementación puedes buscar información de funciones de Haskell como `sort`, `takeWhile`, `dropWhile` o `splitAt`.

4. Recorrido de árboles binarios

Como ya sabes, hay tres tipos de recorridos en profundidad que se pueden utilizar sobre árboles binarios: *pre-order*, *post-order* e *in-order*.

Implementa tres funciones (**preorder t**, **postorder t**, **inorder t**) que, dado un árbol **t**, devuelvan una lista con todos los elementos del árbol en el orden del recorrido correspondiente. Comprueba que las tres funciones funcionan correctamente.

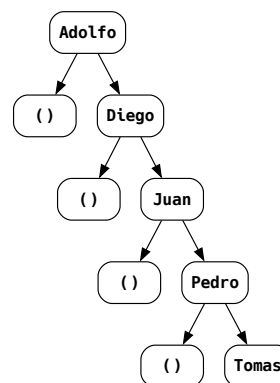
A partir de todas las funciones que ya tienes definidas, implementa una función que equilibre un árbol:

- **balance t** – Construye un árbol equilibrado (es decir, de altura mínima) a partir de otro árbol dado cualquiera.

```
names = build ["Adolfo", "Diego", "Juan", "Pedro", "Tomas"]
```

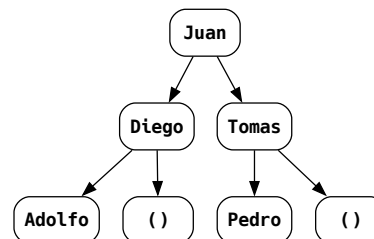
```
print names
```

```
"Adolfo"
\ - ( )
\ - "Diego"
    \ - ( )
    \ - "Juan"
        \ - ( )
        \ - "Pedro"
            \ - ( )
            \ - "Tomas"
```



```
print (balance names)
```

```
"Juan"
\ - "Diego"
    \ - "Adolfo"
    \ - ( )
\ - "Tomas"
    \ - "Pedro"
    \ - ( )
```



5. Búsquedas

El objetivo final de la práctica es poder realizar búsquedas en la estructura de datos que hemos definido. Para ello deberás implementar la siguiente función:

- **between t xmin xmax** – Busca en el árbol binario de búsqueda **t**, y devuelve una lista con todos los elementos del árbol que están entre los valores **xmin** y **xmax** (ambos incluidos).

La búsqueda **debe evitar** recorrer los subárboles que no sea necesario inspeccionar, pero no es necesario que la lista devuelta esté ordenada.

6. Pruebas

Te proporcionamos un fichero **BinaryTreeTest.hs** que prueba todas las funciones solicitadas en la práctica. Además de las pruebas que hagas con el intérprete de Haskell o con tu propio programa principal, **tus funciones deberán compilar con dicho archivo BinaryTreeTest.hs**, sin necesidad de hacer ningún cambio a dicho fichero. **En caso contrario, la calificación de la práctica será de 0.**

Entrega

Como resultado de esta práctica deberás entregar **sólo** el archivo **BinaryTree.hs**, que contenga la definición de la estructura de datos junto con todas las funciones arriba mencionadas, comprimido en un archivo ZIP como se indica más adelante.

Todas las funciones deben tener exactamente el nombre e interfaz definidos en este guión.

No se deben utilizar paquetes, bibliotecas, ni ninguna infraestructura adicional fuera de las bibliotecas estándar del propio lenguaje.

En caso de no compilar siguiendo estas instrucciones, **la nota de la evaluación de la práctica será un 0.**

Todos los archivos de código fuente (y directorios, si es el caso) solicitados en este guión deberán ser comprimidos en un único archivo ZIP con el siguiente nombre:

- **practica6_<nip1>_<nip2>.zip** (donde <nip1> y <nip2> son los NIPs de los estudiantes involucrados) si el trabajo ha sido realizado en pareja.
En este caso sólo uno de los dos estudiantes deberá hacer la entrega en Moodle.
- **practica6_<nip>.zip** (donde <nip> es el NIP del estudiante involucrado) si el trabajo ha sido realizado de forma individual.

El archivo comprimido a entregar no debe contener ningún fichero aparte de los fuentes (y en su caso el fichero `Makefile`) que te pedimos: ningún fichero ejecutable (*.exe) u objeto (*.o), clases compiladas de Java (*.class), ficheros de interfaz de Haskell (*.hi), archivos de configuración del entorno de desarrollo (.vscode), ni ningún otro fichero adicional.

La entrega del archivo ZIP deberá hacerse en la tarea correspondiente preparada en el curso Moodle de la asignatura y antes de la fecha límite fijada para ello.